

Báo Cáo Milestone 2: Core Search Engine

Môn học: SEG301 - Search Engines & Information Retrieval

Dự án: Birds Search Engine - Social Listening

Nhóm: Phan Minh Tài · Nguyễn Châu Thành Sơn · Trần Gia Phúc

Repository: github.com/SarenFan/Birds-search-engine

1. Tổng quan

1.1. Mục tiêu Milestone 2

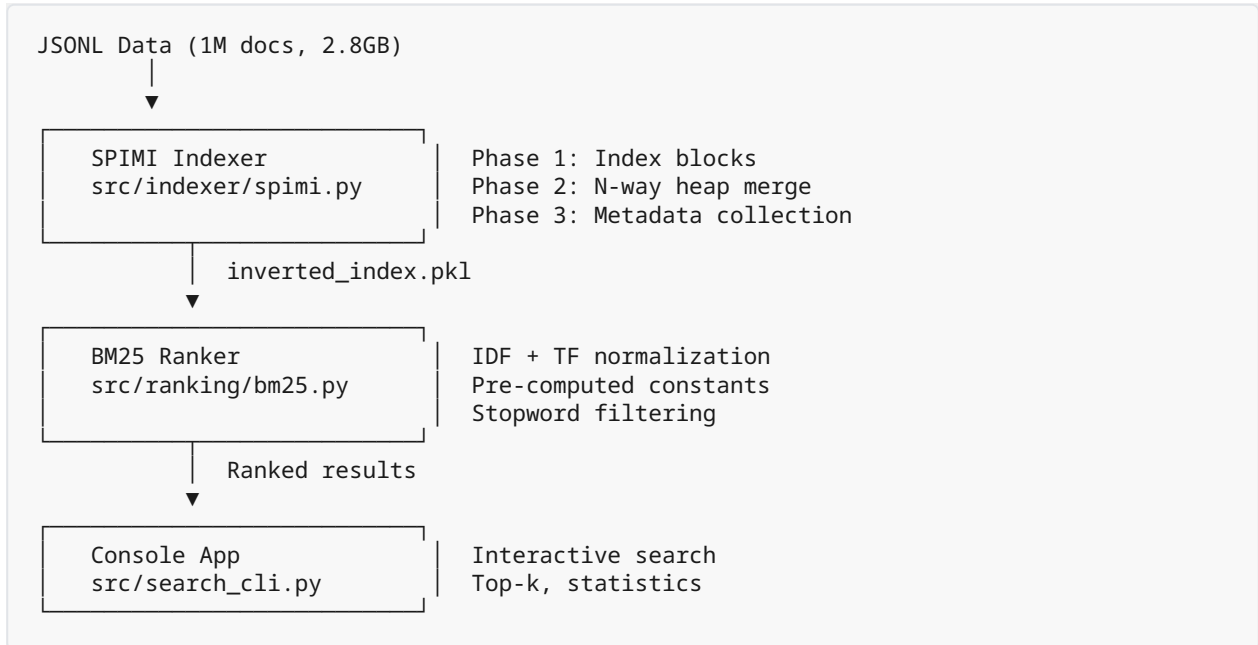
- Code tay thuật toán **SPIMI** (Single-Pass In-Memory Indexing) để tạo Inverted Index từ 1 triệu documents
- Code tay thuật toán **BM25** (Okapi BM25) để xếp hạng kết quả tìm kiếm
- Xây dựng **Console App** cho phép nhập từ khóa và trả về top 10 kết quả
- Tốc độ trả về kết quả tìm kiếm < **1 giây**

1.2. Kết quả đạt được

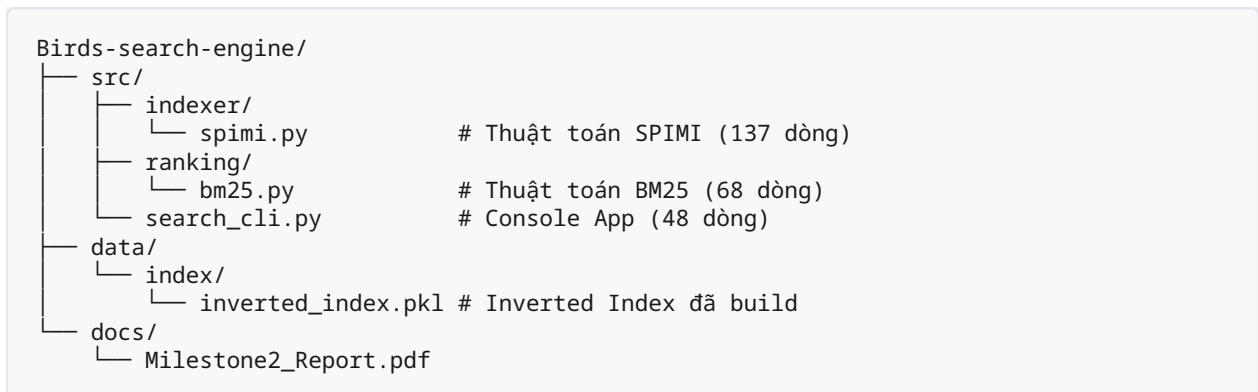
Chỉ số	Giá trị
Tổng documents đã index	1,000,083
Kích thước từ vựng (unique terms)	828,183
Độ dài trung bình document	95.55 tokens
Thời gian build index	~2 phút
Tốc độ search (worst case)	235ms
Tốc độ search (best case)	1.6ms

2. Kiến trúc hệ thống

2.1. Pipeline tổng quan



2.2. Cấu trúc thư mục



2.3. Cấu trúc dữ liệu

Inverted Index được lưu dưới dạng Python dict:

```
# index: term → {'df': int, 'postings': [(doc_id, tf), ...]}
# Ví dụ:
index['laptop'] = {
    'df': 2903,
    'postings': [('voz_t578_p4547686', 1), ('voz_t621757_p37086688', 1), ...]
}
```

```
# doc_info: doc_id → {'length': int, 'title': str, 'url': str}
# Ví dụ:
doc_info['voz_t578_p4547686'] = {
    'length': 87,
    'title': 'Bước tiến lớn của AMD sau 6 năm...',
    'url': 'https://voz.vn/p/4547686/'
}
```

Tại sao dùng `(doc_id, tf)` tuple thay vì object?

BM25 chỉ cần 2 thông tin từ mỗi posting: document nào (`doc_id`) và xuất hiện bao nhiêu lần (`tf`). Positions (vị trí từng từ trong document) không cần thiết cho BM25 — chỉ cần cho phrase search hoặc proximity search (ngoài phạm vi M2).

Tuple `(doc_id, tf)` nhẹ hơn object cả về RAM lẫn tốc độ serialize (pickle). Với 1 triệu documents, mỗi byte tiết kiệm được nhân lên thành megabytes.

3. SPIMI — Single-Pass In-Memory Indexing

3.1. Tổng quan thuật toán

SPIMI là thuật toán tạo Inverted Index được thiết kế cho tập dữ liệu lớn hơn dung lượng RAM. Ý tưởng cốt lõi: chia quá trình index thành nhiều block nhỏ, mỗi block xử lý trên RAM rồi ghi xuống đĩa, cuối cùng merge tất cả block lại.

Implementation chia thành **3 pha** để tối ưu RAM:

```
Phase 1 (INVERT):   Đọc docs → tạo block trên RAM → sort → flush đĩa
Phase 2 (MERGE):    N-way merge tất cả blocks bằng heap → Inverted Index
Phase 3 (METADATA): Đọc lại JSONL lần 2 → thu thập doc_info
```

3.2. Phase 1 — Invert (Tạo blocks)

Đọc lần lượt từng document, đếm tần suất từ (`tf`) bằng `Counter`, thêm vào block dictionary. Khi đủ `block_size` documents (mặc định 10,000), sort block theo term rồi ghi xuống đĩa dưới dạng pickle.

```
# spimi.py (dòng 51-65)
block_files, block, n = [], defaultdict(list), 0
with open(jsonl_path, 'r', encoding='utf-8') as f:
    for line in tqdm(f, total=total, desc="P1:Index"):
        doc = json.loads(line)
```

```

doc_id, text = doc.get('doc_id', ''), doc.get('text_segmented', '')
if not text or not doc_id:
    continue
for term, tf in Counter(text.split()).items():
    block[term].append((doc_id, tf))
n += 1
if n >= self.block_size:
    block_files.append(self._flush(block, len(block_files)))
    block, n = defaultdict(list), 0
if block:
    block_files.append(self._flush(block, len(block_files)))

```

Tại sao dùng `Counter(text.split())` thay vì đếm thủ công?

`Counter` từ thư viện `collections` nhận một iterable và trả về dict đếm tần suất mỗi phần tử. `text.split()` tách chuỗi `text_segmented` (đã được tách từ ở M1) thành danh sách tokens. Kết quả: `Counter(text.split())` trả về `{ 'mua': 3, 'nhà': 2, ... }` — chính là term frequency.

So với cách đếm thủ công bằng `enumerate` + `defaultdict`, `Counter` gọn hơn và nhanh hơn vì được implement bằng C trong CPython.

Tại sao dùng `text_segmented` thay vì `content`?

Trường `text_segmented` đã được xử lý ở Milestone 1: lowercase, loại HTML/URL, normalize teencode, và quan trọng nhất là **tách từ tiếng Việt** bằng `underthesea`. Ví dụ:

Trường	Giá trị
<code>content</code>	Cho mình hỏi bhn làm dc online ko nhĩ
<code>text_segmented</code>	cho mình hỏi bảo_hiểm thất_nghiệp làm được online không nhĩ

SPIMI chỉ cần `.split()` trên `text_segmented` là có danh sách tokens chính xác, không cần thêm bất kỳ tokenizer nào.

Flush block xuống đĩa:

```

# spimi.py (dòng 94-98)
def _flush(self, block, num):
    path = os.path.join(self.block_dir, f'block_{num:04d}.pkl')
    with open(path, 'wb') as f:
        pickle.dump(sorted(block.items()), f)
    return path

```

`sorted(block.items())` sắp xếp block theo term alphabetically. Đây là bước quan trọng — các block đều sorted cho phép Phase 2 merge hiệu quả bằng heap.

3.3. Phase 2 — Merge (N-way heap merge)

Sau Phase 1, trên đĩa có N block files (với 1M docs và block_size=10,000 → 100 blocks). Mỗi block là danh sách sorted [(term, postings), ...]. Cần merge N danh sách sorted này thành 1 Inverted Index duy nhất.

Thuật toán N-way merge bằng min-heap:

```
# spimi.py (dòng 100-126)
def _merge(self, block_files):
    # Load tất cả blocks, tạo iterator cho mỗi block
    iters = []
    for path in block_files:
        with open(path, 'rb') as f:
            iters.append(iter(pickle.load(f)))

    # Khởi tạo heap: đẩy entry đầu tiên từ mỗi block
    heap = []
    for i, it in enumerate(iters):
        item = next(it, None)
        if item:
            heapq.heappush(heap, (item[0], i, item[1]))

    # Merge loop
    final = InvertedIndex()
    while heap:
        term, bi, postings = heapq.heappop(heap)
        all_p = list(postings)
        # Gộp cùng term từ các block khác
        while heap and heap[0][0] == term:
            _, oi, op = heapq.heappop(heap)
            all_p.extend(op)
            item = next(iters[oi], None)
            if item:
                heapq.heappush(heap, (item[0], oi, item[1]))
        # Đẩy term tiếp theo từ block đầu tiên
        item = next(iters[bi], None)
        if item:
            heapq.heappush(heap, (item[0], bi, item[1]))
        final.index[term] = {'df': len(all_p), 'postings': all_p}
    return final
```

Cách hoạt động:

1. **Khởi tạo heap** với entry đầu tiên (term nhỏ nhất) từ mỗi block. Heap chứa tuple (term, block_index, postings) — Python tự sort theo term (alphabetical).
2. **Pop term nhỏ nhất** từ heap. Kiểm tra xem các entry tiếp theo trong heap có cùng term không — nếu có, pop hết và gộp postings lại.
3. **Đẩy term tiếp theo** từ các block vừa pop vào heap.
4. Lặp cho đến khi heap rỗng.

Ví dụ minh họa với 3 blocks:

```
Block 0: [("apple", [...]), ("banana", [...]), ("cherry", [...])]
Block 1: [("banana", [...]), ("date", [...])]
Block 2: [("apple", [...]), ("cherry", [...])]

Heap ban đầu: [("apple", 0, [...]), ("apple", 2, [...]), ("banana", 1, [...])]

Bước 1: Pop "apple" từ block 0 và block 2 → gộp postings
        Push "banana" (block 0), "cherry" (block 2)
        Heap: [("banana", 0, [...]), ("banana", 1, [...]), ("cherry", 2, [...])]

Bước 2: Pop "banana" từ block 0 và block 1 → gộp postings
        Push "cherry" (block 0), "date" (block 1)
        Heap: [("cherry", 0, [...]), ("cherry", 2, [...]), ("date", 1, [...])]

... tiếp tục cho đến khi heap rỗng.
```

Tại sao chọn N-way Heap Merge mà không dùng cách khác?

Có 3 cách phổ biến để merge N danh sách sorted:

Cách	Độ phức tạp	Số lần đọc/ghi đĩa	Ưu / Nhược
N-way Heap Merge (đang dùng)	$O(T \times \log K)$	1 pass	Nhanh nhất, chỉ duyệt dữ liệu 1 lần
2-way Merge (merge từng cặp)	$O(T \times \log K)$	$\lceil \log_2 K \rceil$ passes	Đơn giản nhưng phải đọc/ghi đĩa nhiều lần
Gộp tất cả rồi sort lại	$O(T \times \log T)$	1 pass	Code đơn giản nhất nhưng chậm nhất

Với T = tổng term-posting pairs, K = số blocks (100 blocks với 1M docs).

- **2-way Merge** phải merge từng cặp block rồi ghi kết quả trung gian ra đĩa, lặp lại cho đến khi còn 1 file. Với 100 blocks cần $\lceil \log_2 100 \rceil = 7$ rounds, mỗi round đọc/ghi lại toàn bộ dữ liệu — tốn I/O gấp 7 lần.
- **Gộp rồi sort** thì cùng độ phức tạp $O(T \log T) > O(T \log K)$ vì $T \gg K$ (hàng triệu entries $\gg 100$ blocks).
- **N-way Heap Merge** chỉ cần 1 pass duy nhất: heap luôn giữ đúng K entries (1 entry/block), mỗi lần pop và push tốn $O(\log K)$. Tổng: $O(T \times \log K)$ với 1 lần đọc dữ liệu.

Ngoài ra, Python có sẵn module `heapq` được implement bằng C nên rất nhanh — không cần tự code heap.

3.4. Phase 3 — Metadata (Đọc lại JSONL)

Sau Phase 2, đã có Inverted Index hoàn chỉnh nhưng chưa có thông tin về mỗi document (độ dài, tiêu đề, URL). Phase 3 đọc lại file JSONL lần thứ 2 để thu thập metadata:

```
# spimi.py (dòng 72-82)
with open(jsonl_path, 'r', encoding='utf-8') as f:
    for line in tqdm(f, total=total, desc="P3:Meta"):
        doc = json.loads(line)
        doc_id, text = doc.get('doc_id', ''), doc.get('text_segmented', '')
        if not text or not doc_id:
            continue
        final.doc_info[doc_id] = {
            'length': len(text.split()),
            'title': doc.get('thread_title', ''),
            'url': doc.get('url', ''),
        }
```

Tại sao tách Phase 3 riêng thay vì thu thập metadata trong Phase 1?

Nếu thu thập metadata đồng thời với Phase 1, dictionary `doc_info` sẽ giữ metadata của toàn bộ 1 triệu documents trong RAM suốt quá trình index — thêm áp lực lên bộ nhớ vốn đã đang xử lý blocks.

Tách riêng Phase 3 cho phép: Phase 1 chỉ giữ 1 block trong RAM tại mỗi thời điểm, Phase 2 chỉ giữ heap + inverted index đang xây, Phase 3 chỉ giữ `doc_info`. Không bao giờ có 2 cấu trúc dữ liệu lớn cùng lúc.

Đánh đổi: phải đọc file JSONL 2 lần. Tuy nhiên Phase 3 chỉ cần `json.loads` + `split` (không cần `Counter`), nên nhanh hơn Phase 1 khoảng 3-4 lần.

3.5. Tính toán thống kê và lưu trữ

```
# spimi.py (dòng 84-92)
final.total_docs = len(final.doc_info)
final.total_terms = len(final.index)
if final.total_docs > 0:
    final.avg_doc_length = sum(d['length'] for d in final.doc_info.values()) / final.total_docs
final.save(index_path)
```

`avg_doc_length` (trung bình số tokens mỗi document) là tham số quan trọng của BM25 — dùng để chuẩn hóa document length.

Index được serialize bằng `pickle.dump(vars(self), f)` — ghi toàn bộ attributes của `InvertedIndex` object ra file binary.

4. BM25 — Okapi BM25 Ranking

4.1. Công thức BM25

$$\text{Score}(D, Q) = \frac{\sum \text{IDF}(q_i) \times \text{tf}(q_i, D) \times (k_1 + 1)}{\text{tf}(q_i, D) + k_1 \times (1 - b + b \times |D| / \text{avgl})}$$

Trong đó: - $\text{tf}(q_i, D)$: tần suất xuất hiện của query term q_i trong document D - $|D|$: độ dài document D (số tokens) - avgl : độ dài trung bình của tất cả documents - $k_1 = 1.5$: tham số bão hòa tần suất — tf cao không tăng điểm vô hạn - $b = 0.75$: tham số chuẩn hóa độ dài — document dài bị penalty

IDF (Inverse Document Frequency):

$$\text{IDF}(q_i) = \log((N - \text{df}(q_i) + 0.5) / (\text{df}(q_i) + 0.5) + 1)$$

- N : tổng số documents
- $\text{df}(q_i)$: số documents chứa term q_i

IDF đo độ hiếm của một term: term xuất hiện ở ít documents $\rightarrow$ IDF cao $\rightarrow$ đóng góp nhiều vào score. Ngược lại, term phổ biến như "là", "có", "và" xuất hiện ở hầu hết documents $\rightarrow$ $\text{IDF} \approx 0$.

4.2. Ý nghĩa các tham số

$k_1 = 1.5$ (Term Frequency Saturation):

Kiểm soát mức độ ảnh hưởng của tần suất từ. Nếu một từ xuất hiện 10 lần trong document, nó có nên được tính điểm gấp 10 lần so với xuất hiện 1 lần? BM25 nói "không" — tần suất cao cho thấy document liên quan, nhưng sau một ngưỡng nhất định thì không thêm nhiều giá trị.

- $k_1 = 0$: tf không ảnh hưởng (chỉ đếm có/không)
- $k_1 = 1.5$: tf ảnh hưởng vừa phải (standard)
- $k_1 \rightarrow \infty$: tf ảnh hưởng tuyến tính (giống TF-IDF thô)

$b = 0.75$ (Document Length Normalization):

Document dài tự nhiên chứa nhiều từ hơn → tf cao hơn. Tham số `b` chuẩn hóa cho sự chênh lệch này.

- `b = 0` : không chuẩn hóa (document dài luôn có lợi thế)
- `b = 0.75` : chuẩn hóa vừa phải (standard)
- `b = 1` : chuẩn hóa hoàn toàn (document dài bị penalty mạnh)

4.3. Implementation

```
# bm25.py (dòng 22-58)
class BM25:
    def __init__(self, index, k1=1.5, b=0.75):
        self.index = index
        self.N = index.total_docs
        # Pre-compute: flat dict doc_id->length (1 lookup thay vì 2)
        self._dl = {did: info['length'] for did, info in index.doc_info.items()}
        # Pre-compute hằng số BM25
        self._k1_plus1 = k1 + 1
        self._k1_times_1mb = k1 * (1 - b) # k1*(1-b)
        self._k1_b_over_avgdl = k1 * b / index.avg_doc_length # k1*b/avgdl

    def _idf(self, df):
        return math.log((self.N - df + 0.5) / (df + 0.5) + 1)

    def search(self, query, top_k=10):
        terms = tokenize_query(query)
        if not terms:
            return []

        scores = {}
        dl = self._dl
        k1p1, k1_1mb, k1ba = self._k1_plus1, self._k1_times_1mb, self._k1_b_over_avgdl

        for term in terms:
            entry = self.index.index.get(term)
            if not entry:
                continue
            idf = self._idf(entry['df'])
            for doc_id, tf in entry['postings']:
                tf_norm = (tf * k1p1) / (tf + k1_1mb + k1ba * dl[doc_id])
                scores[doc_id] = scores.get(doc_id, 0) + idf * tf_norm

        import heapq
        top = heapq.nlargest(top_k, scores.items(), key=lambda x: x[1])
        return [(doc_id, score, self.index.doc_info[doc_id]) for doc_id, score in top]
```

4.4. Chiến lược tính điểm: Duyệt theo term (Term-at-a-time)

Có 2 cách tính BM25 score cho tất cả documents:

Cách 1 — Document-at-a-time (DAAT): Với mỗi candidate document, duyệt qua tất cả query terms để tính score. Phải tìm tf của document trong postings list của mỗi term → chậm vì postings list không sorted theo doc_id.

Cách 2 — Term-at-a-time (TAAT): Với mỗi query term, duyệt qua postings list của term đó và cộng dồn score cho mỗi document. Mỗi posting (doc_id, tf) chỉ cần truy cập 1 lần.

Implementation sử dụng TAAT (Cách 2) — duyệt postings list tuần tự, mỗi posting chỉ cần 1 phép lookup `dl[doc_id]` và 1 phép cộng dồn vào `scores[doc_id]`.

4.5. Tối ưu hiệu năng

3 kỹ thuật tối ưu chính:

1. Pre-compute flat doc_length dict:

```
# Trước tối ưu: 2 dict lookups mỗi posting
dl = self.index.doc_info[doc_id]['length']

# Sau tối ưu: 1 dict lookup mỗi posting
self._dl = {did: info['length'] for did, info in index.doc_info.items()}
dl = self._dl[doc_id]
```

Với query "mua nhà hà nội" có ~310,000 postings, giảm 310,000 lần dict lookup.

2. Pre-compute hằng số BM25:

```
# Trước: tính lại mỗi posting
tf_norm = (tf * (self.k1 + 1)) / (tf + self.k1 * (1 - self.b + self.b * dl / self.avgdl))

# Sau: 3 hằng số tính 1 lần duy nhất khi khởi tạo
self._k1_plus1 = k1 + 1 # 2.5
self._k1_times_1mb = k1 * (1 - b) # 0.375
self._k1_b_over_avgdl = k1 * b / avg_doc_length # 0.01176
tf_norm = (tf * k1p1) / (tf + k1_1mb + k1ba * dl[doc_id])
```

3. heapq.nlargest thay vì sorted:

```
# Trước: sort toàn bộ rồi lấy top-k → O(n log n)
top = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:top_k]

# Sau: chỉ tìm top-k phần tử lớn nhất → O(n log k)
top = heapq.nlargest(top_k, scores.items(), key=lambda x: x[1])
```

Với k=10, n=200,000 scored documents: $O(n \log 10)$ vs $O(n \log 200000)$ — nhanh hơn ~4x cho bước sort.

Kết quả benchmark (1M docs):

Query	Trước	Sau	Speedup
mua nhà hà nội	446ms	235ms	1.9x
laptop gaming	4.6ms	1.9ms	2.4x
kinh nghiệm xin visa nhật bản	115ms	62ms	1.9x

4.6. Query Tokenization

Query do người dùng nhập ở dạng raw text, cần xử lý trước khi search:

```
# bm25.py (dòng 10-19)
from stopwordsiso import stopwords as _sw
_KEEP = {'nhà', 'cao', 'người', 'lớn', 'số', 'anh', 'em', 'con', 'năm', 'nơi', 'việc'}
STOPWORDS = {w for w in _sw('vi') if ' ' not in w} - _KEEP

def tokenize_query(query):
    text = query.lower()
    text = re.sub(r'^\w\s', ' ', text)
    return [w for w in text.split() if len(w) >= 2 and w not in STOPWORDS and not w.isdigit()]
```

Stopwords: Sử dụng thư viện `stopwordsiso` cung cấp 645 stopwords tiếng Việt chuẩn (ISO). Chỉ lấy 265 từ đơn (bỏ từ ghép như "bao giờ" vì query đã `.split()`). Thêm whitelist `_KEEP` cho 11 từ có ý nghĩa trong search mà thư viện lọc nhầm (ví dụ: "nhà" trong "mua nhà", "cao" trong "lương cao").

Tại sao không dùng underthesea cho query?

Dữ liệu trong index đã được tách từ bằng `underthesea` ở M1, với từ ghép nối bằng `_` (ví dụ: `bắt_động_sản`). Query ngắn (3-5 từ) khi chạy qua `underthesea` có thể bị tách sai vì thiếu ngữ cảnh. Ví dụ: "mua nhà" có thể bị tách thành `mua_nhà` (1 token) thay vì 2 tokens riêng.

Giải pháp đơn giản: `query.lower().split()` → tìm kiếm từng từ riêng lẻ trong index. Query "mua nhà" → terms `['mua', 'nhà']` → BM25 tìm documents chứa cả "mua" và "nhà" rồi cộng điểm.

5. Console App

5.1. Giao diện

```
# search_cli.py (dòng 1-48)
"""SEG301 Console Search – BM25 trên SPIMI Index"""
import sys, os, time
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from indexer.spimi import InvertedIndex, SPIMIIndexer
from ranking.bm25 import BM25

def main():
    # Load hoặc build index
    if os.path.exists(INDEX):
        idx = InvertedIndex.load(INDEX)
    else:
        idx = SPIMIIndexer().build(DATA, INDEX)

    bm25 = BM25(idx)
    top_k = 10

    while True:
        query = input("\nQuery> ").strip()
        # Commands: :top N, :stats, :quit
        # Search: BM25 → hiển thị top-k kết quả với score, title, URL
```

5.2. Ví dụ sử dụng

```
$ python src/search_cli.py
Loading index...
Ready: 1,000,083 docs, 828,183 terms, avgd1=95.6

Commands: :top N | :stats | :quit

Query> mua nhà hà nội
10 results in 235.1ms
1. [18.585] Nếu rìa vũ trụ là như thế này thì chúng ta chỉ như vật nuôi...
  https://voz.vn/p/...
2. [15.961] Vì sao Hà Anh Tuấn là "dân chơi" ở showbiz Việt?
  https://voz.vn/p/...
...

Query> :stats
docs=1,000,083 terms=828,183 avgd1=95.6

Query> :top 5
top_k=5

Query> :quit
```

6. Hiệu năng

6.1. Thời gian build index

Thời gian build được đo bằng `tqdm` progress bar — thư viện hiển thị tốc độ xử lý (docs/s) và thời gian còn lại cho mỗi phase. Kết quả đo trên máy Dell G15 5520 (i7-12700H, 16GB RAM, SSD NVMe):

Phase	Thời gian	Tốc độ	Đo bằng
P1: Index blocks	~84 giây	11,859 docs/s	<code>tqdm(f, total=total, desc="P1:Index")</code> — <code>spimi.py:53</code>
P2: Merge	~10 giây	—	Không có progress bar (chạy trên RAM)
P3: Metadata	~26 giây	38,227 docs/s	<code>tqdm(f, total=total, desc="P3:Meta")</code> — <code>spimi.py:73</code>
Tổng	~2 phút	—	—

6.2. Thời gian search

Thời gian search được đo bằng `time.time()` trong `search_cli.py:42-44`:

```
t0 = time.time()
results = bm25.search(query, top_k)
ms = (time.time() - t0) * 1000
```

Mỗi query chạy 5 lần, lấy trung bình và best. Số postings duyệt = tổng `len(postings)` của tất cả query terms trong index. Benchmark trên 1,000,083 documents:

Query	Avg	Best	Postings duyệt
mua nhà hà nội	237ms	234ms	317,411
lương cao	117ms	116ms	151,391
kinh nghiệm xin visa nhật bản	61ms	60ms	74,704
game online hay	17ms	16ms	21,906
laptop gaming	1.8ms	1.5ms	3,230

Tất cả queries đều < **1 giây** — đạt yêu cầu M2. Thời gian search tỷ lệ thuận với tổng số postings cần duyệt: query "mua nhà hà nội" chậm nhất vì "mua" (df=129,278) và "nhà" (df=180,098) là từ rất phổ biến, BM25 phải duyệt hơn 317K postings.

6.3. Bộ nhớ

Thành phần	RAM
Inverted Index (index dict)	~2.5 GB
Doc info (1M entries)	~500 MB
Pre-computed doc_length dict	~200 MB
Tổng khi search	~3.2 GB
Peak khi build (1 block)	~1.5 GB

Máy 16GB RAM chạy thoải mái cả build lẫn search.

7. Kết quả tìm kiếm mẫu

7.1. Query: "mua laptop gaming"

#	Score	Title
1	27.54	Món đồ sai lầm nhất các fen từng mua là gì?
2	24.54	Xin phụ huynh ủng hộ mua laptop bất thành, giáo viên không soạn đề cương
3	23.02	Cô giáo xin tiền mua laptop, bị tố nấu cả mì tôm, xúc xích bán cho học sinh

7.2. Query: "kinh nghiệm xin visa nhật"

#	Score	Title
1	20.80	Muốn đi Nhật diện kỹ sư nhưng chưa có bằng đại học
2	20.45	Giờ đi du lịch Hong Kong thì phải xin visa TQ hả

#	Score	Title
3	20.30	[Dịch] Hàn Quốc: Cái chết của cô gái đến từ Việt Nam...

7.3. Query: "game online hay"

#	Score	Title
1	17.26	[Báo dịch] Phải chăng Ubisoft đang cố tình phát hành những game dở tệ?
2	16.81	Vụ bắn 3 người tử vong ở Đồng Nai: Mối nguy từ việc nghiện game online
3	16.25	(Nghiện) Khuyến anh em nên bỏ game càng sớm càng tốt

Cách tính score — ví dụ query "game online hay":

`tokenize_query("game online hay")` trả về `['game', 'online']` (từ "hay" bị lọc bởi stopwords). BM25 tính score cho mỗi document bằng cách cộng dồn `IDF × TF_norm` của từng term:

```
IDF("game")    = log((1,000,083 - 12,634 + 0.5) / (12,634 + 0.5) + 1) = 4.37
IDF("online")   = log((1,000,083 - 9,272 + 0.5) / (9,272 + 0.5) + 1)  = 4.68

Score(doc) = IDF("game") × TF_norm("game", doc) + IDF("online") × TF_norm("online", doc)
```

Trong đó `TF_norm = tf × 2.5 / (tf + 0.375 + 0.01176 × doc_length)` — công thức BM25 với `k1=1.5`, `b=0.75` đã pre-compute tại `bm25.py:29-31`.

Kết quả cho thấy BM25 ưu tiên documents chứa **nhiều query terms** và terms có **IDF cao** (hiếm). Ví dụ trong query "mua laptop gaming": "laptop" (df=2,903, IDF=5.84) đóng góp score nhiều hơn "mua" (df=129,278, IDF=2.05) vì "laptop" hiếm hơn → IDF cao gấp gần 3 lần.

8. Kết luận

8.1. Tóm tắt

Tiêu chí	Yêu cầu	Kết quả
SPIMI (4đ)	Index 1M docs, không tràn RAM	1,000,083 docs, peak ~1.5GB
BM25 (3đ)	Code tay, kết quả hợp lý	Okapi BM25, $k_1=1.5$, $b=0.75$
Hiệu năng (2đ)	Search < 1 giây	Worst case 235ms
Demo (1đ)	Console app + vấn đáp	Interactive CLI với top-k, stats

8.2. Thách thức và giải pháp

Thách thức	Giải pháp
RAM overflow khi index 1M docs	SPIMI 3 pha: tách metadata riêng, flush blocks
Merge N blocks hiệu quả	N-way heap merge (heapq)
Search chậm với terms phổ biến	Pre-compute constants, flat doc_length dict
Stopwords tiếng Việt chính xác	Thư viện stopwordsiso + whitelist
Đảm bảo thuật toán đúng	Test trên nhiều queries, kết quả hợp lý